

EXP 2

Aim: Create a Blockchain using Python

Theory:

Blockchain is a simplified ledger that stores data in a sequence of linked blocks. Each block contains transactions, a timestamp, proof from Proof-of-Work, and the hash of the previous block. In this program, users can enter transactions such as **"5 BTC transferred from Alice to Bob"**, which are stored as pending transactions and added to the blockchain when a block is mined.

In this program:

1. **Transaction Creation** – Users enter transactions, which are temporarily stored as pending transactions.
2. **Block Creation** – `create_block()` creates a new block containing the transactions, timestamp, proof, and previous block hash.
3. **Proof-of-Work (PoW)** – The program finds a proof whose SHA-256 hash starts with `0000` before a block can be mined.
4. **Hashing** – SHA-256 is used to generate hashes and link each block to the previous block.
5. **Validation** – `is_chain_valid()` checks the previous hashes and Proof-of-Work to determine whether the blockchain is valid.
6. **Tampering** – The `/tamper` route intentionally changes a transaction to demonstrate how modifying a block makes the blockchain invalid.
7. **Flask Web API** – Routes `/add_transaction`, `/mine`, `/chain`, `/is_valid`, and `/tamper` allow users to add transactions, mine blocks, view the blockchain, check its validity, and demonstrate tampering.

Flask Web API

The program provides the following endpoints:

- `/add_transaction` → add a new transaction to the pending transaction list
- `/mine` → mine pending transactions into a new block
- `/chain` → display the complete blockchain
- `/is_valid` → check whether the blockchain is valid
- `/tamper` → intentionally modify a block to demonstrate invalidation

The complete working of the program can be summarized as:

User enters transaction
↓
Transaction stored as pending
↓
Mine the transaction
↓
Proof-of-Work is performed
↓
New block is created
↓
Previous block hash is stored
↓
Block is added to blockchain
↓
Blockchain can be validated
↓
Tampering changes block data
↓
Hash mismatch is detected
↓
Blockchain becomes INVALID

Code:

```
import hashlib
import json
from datetime import datetime
from flask import Flask, jsonify, request

class Blockchain:

    def __init__(self):
        self.chain = []
        self.pending_transactions = []

        # Genesis block
        self.create_block(1, "0", ["Genesis Block"])
```

```

def create_block(self, proof, previous_hash, transactions):
    block = {
        "index": len(self.chain) + 1,
        "timestamp": str(datetime.now()),
        "transactions": transactions,
        "proof": proof,
        "previous_hash": previous_hash
    }

    self.chain.append(block)
    return block

def get_last_block(self):
    return self.chain[-1]

def hash(self, block):
    data = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(data).hexdigest()

def proof_of_work(self, previous_proof):
    proof = 1

    while True:
        value = proof**2 - previous_proof**2
        hash_value = hashlib.sha256(str(value).encode()).hexdigest()

        if hash_value.startswith("0000"):
            return proof

        proof += 1

def add_transaction(self, transaction):
    self.pending_transactions.append(transaction)

def mine_block(self):
    previous_block = self.get_last_block()

    proof = self.proof_of_work(previous_block["proof"])
    previous_hash = self.hash(previous_block)

    block = self.create_block(
        proof,
        previous_hash,

```

```

        self.pending_transactions.copy()
    )

    self.pending_transactions.clear()

    return block

def is_chain_valid(self):
    for i in range(1, len(self.chain)):

        previous = self.chain[i - 1]
        current = self.chain[i]

        # Check previous hash
        if current["previous_hash"] != self.hash(previous):
            return False

        # Check proof of work
        value = current["proof"]**2 - previous["proof"]**2
        hash_value = hashlib.sha256(str(value).encode()).hexdigest()

        if not hash_value.startswith("0000"):
            return False

    return True

# -----
# Flask App
# -----

app = Flask(__name__)
blockchain = Blockchain()

# Add a transaction
@app.route("/add_transaction")
def add_transaction():

    transaction = request.args.get("transaction")

    if not transaction:
        return "Please provide a transaction."

```

```

blockchain.add_transaction(transaction)

return jsonify({
    "message": "Transaction added successfully.",
    "transaction": transaction
})

# Mine pending transactions
@app.route("/mine")
def mine():

    if not blockchain.pending_transactions:
        return jsonify({
            "message": "No transactions to mine."
        }), 400

    block = blockchain.mine_block()

    return jsonify({
        "message": "Block mined successfully.",
        "block": block
    })

# View blockchain
@app.route("/chain")
def get_chain():

    return jsonify({
        "length": len(blockchain.chain),
        "chain": blockchain.chain
    })

# Check validity
@app.route("/is_valid")
def is_valid():

    valid = blockchain.is_chain_valid()

    return jsonify({
        "valid": valid,
        "message": "Blockchain is VALID."
    })

```

```

        if valid else
        "Blockchain is NOT VALID."
    })

# Intentionally modify a block
@app.route("/tamper")
def tamper():

    if len(blockchain.chain) < 2:
        return jsonify({
            "message": "Mine a block first."
        }), 400

    blockchain.chain[1]["transactions"][0] = (
        "HACKED: 100 BTC transferred to Hacker"
    )

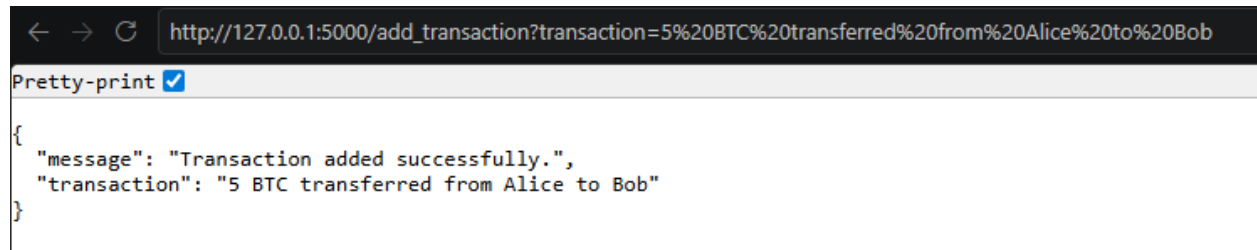
    return jsonify({
        "message": "Blockchain has been tampered with!"
    })

if __name__ == "__main__":
    app.run(debug=True)

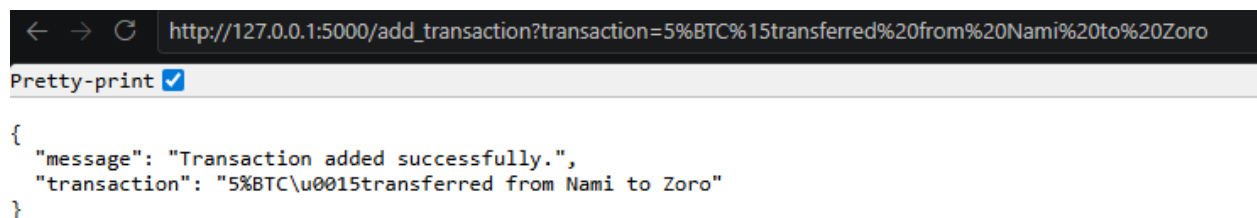
```

Output:

Adding Transactions:



A screenshot of a web browser window. The address bar shows the URL: `http://127.0.0.1:5000/add_transaction?transaction=5%20BTC%20transferred%20from%20Alice%20to%20Bob`. Below the address bar, there is a checkbox labeled "Pretty-print" which is checked. The main content area displays a JSON response: `{ "message": "Transaction added successfully.", "transaction": "5 BTC transferred from Alice to Bob" }`.



A screenshot of a web browser window. The address bar shows the URL: `http://127.0.0.1:5000/add_transaction?transaction=5%BTC%15transferred%20from%20Nami%20to%20Zoro`. Below the address bar, there is a checkbox labeled "Pretty-print" which is checked. The main content area displays a JSON response: `{ "message": "Transaction added successfully.", "transaction": "5%BTC\u0015transferred from Nami to Zoro" }`.

```
← → ↻ http://127.0.0.1:5000/add_transaction?transaction=5%10BTC%20transferred%20from%20Vivi%20to%20Robin
Pretty-print ☐
{
  "message": "Transaction added successfully.",
  "transaction": "5\u0010BTC transferred from Vivi to Robin"
}
```

Mining block:

```
← → ↻ http://127.0.0.1:5000/mine
Pretty-print ☒
{
  "block": {
    "index": 2,
    "previous_hash": "fb9752a1bd42577c6f0959bd3cfe15bac38fe1d222927d6ec4175e1320b808bd",
    "proof": 533,
    "timestamp": "2026-08-19 12:33:49.648305",
    "transactions": [
      "5 BTC transferred from Alice to Bob",
      "5%BTC\u0015transferred from Nami to Zoro",
      "5\u0010BTC transferred from Vivi to Robin"
    ]
  },
  "message": "Block mined successfully."
}
```

For block 3:

```
← → ↻ http://127.0.0.1:5000/add_transaction?transaction=25%20BTC%20transferred%20from%20Sanji%20to%20Chopper
Pretty-print ☒
{
  "message": "Transaction added successfully.",
  "transaction": "25 BTC transferred from Sanji to Chopper"
}
```

```
← → ↻ http://127.0.0.1:5000/mine
Pretty-print ☒
{
  "block": {
    "index": 3,
    "previous_hash": "37413caf2d2a016b953238941e0844718c63b7ba09f17e6735912d6b8dca7b8d",
    "proof": 45293,
    "timestamp": "2026-08-19 12:37:27.756148",
    "transactions": [
      "5\u0010BTC transferred from Vivi to Robin",
      "25 BTC transferred from Sanji to Chopper"
    ]
  },
  "message": "Block mined successfully."
}
```

For block 4:

```
← → ↻ http://127.0.0.1:5000/add_transaction?transaction=13%20BTC%20transferred%20from%20Jimbei%20to%20Usopp
Pretty-print ☒
```

```
{
  "message": "Transaction added successfully.",
  "transaction": "13 BTC transferred from Jimbei to Usopp"
}
```

```
← → ↻ http://127.0.0.1:5000/mine
Pretty-print ☐
```

```
{
  "block": {
    "index": 4,
    "previous_hash": "16ee3b551662b3e68264e33ab25844a66b56601fef745f77978c061f4b96f0d8",
    "proof": 21391,
    "timestamp": "2026-08-19 12:39:43.833365",
    "transactions": [
      "13 BTC transferred from Jimbei to Usopp"
    ]
  },
  "message": "Block mined successfully."
}
```

Final blockchain of 4 blocks:

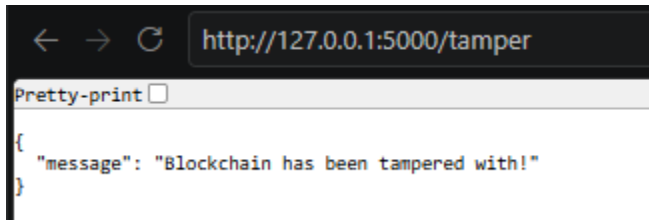
```
← → ↻ http://127.0.0.1:5000/chain
Pretty-print ☐
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-08-19 12:29:39.138365",
      "transactions": [
        "Genesis Block"
      ]
    },
    {
      "index": 2,
      "previous_hash": "fb9752a1bd42577c6f0959bd3cfe15bac38fed222927d6ec4175e1320b808bd",
      "proof": 533,
      "timestamp": "2026-08-19 12:33:49.648305",
      "transactions": [
        "5 BTC transferred from Alice to Bob",
        "5 BTC transferred from Nami to Zoro",
        "5 BTC transferred from Vivi to Robin"
      ]
    },
    {
      "index": 3,
      "previous_hash": "37413caf2d2a016b953238941e0844718c63b7ba09f17e6735912d6b8dca7b8d",
      "proof": 45293,
      "timestamp": "2026-08-19 12:37:27.756148",
      "transactions": [
        "5 BTC transferred from Vivi to Robin",
        "25 BTC transferred from Sanji to Chopper"
      ]
    },
    {
      "index": 4,
      "previous_hash": "16ee3b551662b3e68264e33ab25844a66b56601fef745f77978c061f4b96f0d8",
      "proof": 21391,
      "timestamp": "2026-08-19 12:39:43.833365",
      "transactions": [
        "13 BTC transferred from Jimbei to Usopp"
      ]
    }
  ],
  "length": 4
}
```

Check is_valid:

```
← → ↻ http://127.0.0.1:5000/is_valid
Pretty-print ☐
{
  "message": "Blockchain is VALID.",
  "valid": true
}
```

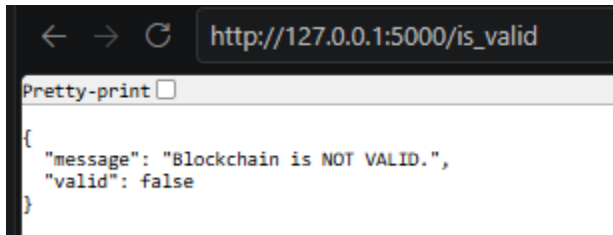
To create an invalid blockchain scenario:

Tamper:

A screenshot of a web browser window. The address bar shows the URL 'http://127.0.0.1:5000/tamper'. Below the address bar, there is a 'Pretty-print' checkbox. The main content area displays a JSON object: { "message": "Blockchain has been tampered with!" }.

```
← → ↻ http://127.0.0.1:5000/tamper
Pretty-print ☐
{
  "message": "Blockchain has been tampered with!"
}
```

Blockchain isn't valid:

A screenshot of a web browser window. The address bar shows the URL 'http://127.0.0.1:5000/is_valid'. Below the address bar, there is a 'Pretty-print' checkbox. The main content area displays a JSON object: { "message": "Blockchain is NOT VALID.", "valid": false }.

```
← → ↻ http://127.0.0.1:5000/is_valid
Pretty-print ☐
{
  "message": "Blockchain is NOT VALID.",
  "valid": false
}
```

Conclusion

The experiment successfully demonstrated the basic working of a blockchain by creating blocks, adding transactions, performing Proof-of-Work, and linking blocks using SHA-256 hashes. The validation and tampering features also showed how blockchain can detect changes and maintain data integrity.